

Ejercicio 3: Patrones de diseño aplicados en la vida real.

Como primer ejemplo en donde se podría aplicar un patrón de diseño en concreto es en el patrón "Builder", llevándolo a la vida real podríamos aplicar este patrón por ejemplo a la hora de querer construir una casa.

Para ejecutar esto el problema está en levantar una propiedad y gestionar el acopio de materiales ya que esto requiere una secuencia de pasos muy específica. No se puede poner el techo sin antes hacer los cimientos y levantar las paredes. Intentar hacer todo de golpe o sin un orden modular resulta en un caos logístico.

La solución con el patrón Builder es que te permite construir objetos (o proyectos) complejos paso a paso. Este patrón te asegura que el mismo proceso de construcción se respete en orden, sin importar si los materiales que se acopiaron cambian en el camino.

Otro ejemplo en concreto en el que se podría aplicar otro patrón de diseño es con "State", ya que por ejemplo en un restaurante que tiene que gestionar las órdenes automatizadas, implica lidiar con pedidos que cambian constantemente: Recibido, En Preparación, Enviado o Entregado. Si un cliente intenta cancelar por WhatsApp un pedido que ya está en la plancha, el sistema debe impedirlo. Manejar esto con condicionales (if/else) hace que el código de la plataforma sea inmanejable a medida que la vas escalando. Aplicando State, permitís que un objeto (el pedido) altere su comportamiento cuando su estado interno cambia. Al aislar cada fase en su propia clase, la lógica fluye sola: el bot sabe exactamente cómo reaccionar a un mensaje del cliente dependiendo del estado actual de esa hamburguesa, manteniendo tu código limpio y modular.

Un ejemplo donde podríamos aplicar el patrón de diseño "Observer" es en las notificaciones de expensas y asambleas de un edificio.

El problema es que en la gestión del edificio, cuando se carga una nueva liquidación o se abre en una votación en una aplicación por ej "REDCONAR", los propietarios necesitan enterarse. Que cada vecino tenga que entrar a la plataforma todos los días para verificar si hay novedades es un proceso ineficiente.

El patrón Observer define un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento. En este caso, la administración del edificio es el sujeto a observar y los vecinos son los suscriptores. Al subir un nuevo documento o acta, el sistema "dispara" automáticamente la notificación a los correos o celulares

de todos los propietarios suscritos, automatizando el flujo de información